

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Тема: «Поиск набора подстрок в строке.
(Ахо–Корасик)»
Вариант 2

Студент гр. 4343

Стариков Н.С.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Задание

1. Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст (T , $1 \leq |T| \leq 100000$).

Вторая строка содержит число n ($1 \leq n \leq 3000$), каждая следующая из n строк содержит шаблон из набора $P = \{p_1, \dots, p_n\}$ ($1 \leq |p_i| \leq 75$).

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$.

Выход:

Все вхождения образцов из P в T .

Каждое вхождение образца в текст представить в виде двух чисел - i и p , где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

2. Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с джокером.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблону образцу P необходимо найти все вхождения P в текст T .

Например, образец $ab??c?$ с джокером $?$ встречается дважды в тексте $xabvccbababcsax$.

Символ джокер не входит в алфавит, символы которого используются в T . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида $???$ недопустимы. Все строки содержат символы из алфавита $\{A, C, G, T, N\}$.

Вход:

Текст T ($1 \leq |T| \leq 100000$)

Шаблон P ($1 \leq |P| \leq 40$)

Символ джокера.

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер). Номера должны выводиться в порядке возрастания.

Индивидуализация:

Вариант 2. Подсчитать количество вершин в автомате; вывести список найденных образцов, имеющих пересечения с другими найденными образцами в строке поиска.

Описание алгоритмов и оценка их сложности во времени и памяти

В первой задаче реализуется алгоритм Ахо-Корасик для точного множественного поиска набора образцов в тексте. Входные данные: текст T длины до 100000, число образцов n до 3000, каждый образец длиной до 75 символов из алфавита $\{A, C, G, T, N\}$. Алгоритм состоит из трёх этапов: построение бора (префиксного дерева) из всех образцов, построение автомата путём добавления суффиксных ссылок (fail) и выходных (сжатых) ссылок (output), а затем поиск в тексте с использованием построенного автомата. Построение бора занимает время $O(M)$, где M = сумма длин всех образцов, причём $M \leq 3000 \cdot 75 = 225000$. На каждой вершине бора хранятся переходы по 5 символам, что даёт объём памяти $O(M \cdot |\Sigma|) = O(M \cdot 5) = O(M)$. После построения бора выполняется обход в ширину для установки fail-ссылок и output-ссылок: для каждой вершины за $O(|\Sigma|)$ обрабатываются переходы, что суммарно даёт $O(M \cdot |\Sigma|)$. Важно, что в автомате используются выходные ссылки, позволяющие за $O(1)$ переходить к ближайшей терминальной вершине, а не подниматься по цепочке fail-ссылок, что гарантирует эффективность. Поиск в тексте длины $|T|$ выполняется за $O(|T| + \text{total_matches})$, где total_matches — общее количество найденных вхождений. Каждый символ текста обрабатывается за $O(1)$ — переход по автомату и проверка выходных ссылок. Поскольку выходные ссылки ведут только к вершинам, содержащим образцы, суммарное время на обработку всех выходных ссылок пропорционально количеству найденных вхождений. В худшем случае количество вхождений может быть $O(|T| \cdot (\text{средняя длина образца}))$, но с учётом ограничений (длина образца до 75, $|T|$ до 100000) это приемлемо. По памяти алгоритм требует $O(M \cdot |\Sigma|)$ для хранения автомата (переходы, fail, output, списки образцов) и $O(|T|)$ для хранения строки и результатов (хотя ответы можно выводить на лету).

Итоговая временная сложность — $O(M \cdot |\Sigma| + |T| + \text{total_matches})$, что на практике линейно относительно входных данных.

Память — $O(M \cdot |\Sigma| + |T|)$.

Во второй задаче необходимо найти все вхождения шаблона с джокерами (wildcard) в текст. Шаблон содержит специальный символ (джокер), совпадающий с любым символом из алфавита $\{A, C, G, T, N\}$. Длина текста $|T| \leq 100000$, длина шаблона $|P| \leq 40$, в шаблоне есть хотя бы один не-джокер. Алгоритм использует тот же подход Ахо-Корасик, но применяет его к сегментам шаблона, разделённым джокерами. Сначала шаблон разбивается на максимальные непрерывные подстроки, не содержащие джокеров. Каждый такой сегмент имеет длину не более $|P|$, а общее количество сегментов $k \leq |P|$. Суммарная длина всех сегментов $M \leq |P| \leq 40$. Для этих сегментов строится автомат Ахо-Корасик (бор с суффиксными и выходными ссылками) аналогично первой задаче. Время построения автомата — $O(M \cdot |\Sigma|) = O(40 \cdot 5) = O(1)$ практически. Затем выполняется проход по тексту: для каждой позиции pos в тексте (0-индексация) находится текущее состояние автомата и через выходные ссылки определяются все сегменты, которые заканчиваются в этой позиции. Для каждого такого сегмента с индексом id вычисляется потенциальное начало шаблона в тексте: $start = pos - \text{len}(\text{seg}[id]) + 1 - \text{offset}[id]$, где $\text{offset}[id]$ — позиция сегмента в исходном шаблоне. Если $start \geq 0$ и $start + |P| \leq |T|$, то увеличивается счётчик совпадений для этой стартовой позиции. После обработки всего текста для каждой стартовой позиции проверяется, равно ли количество совпавших сегментов общему числу сегментов k . Если да, то позиция ($start+1$, в 1-индексации) добавляется в ответ. Временная сложность поиска: обработка каждого символа текста требует $O(1)$ переходов по автомату и прохода по цепочке выходных ссылок. Поскольку каждый сегмент может быть найден в нескольких позициях, общее количество найденных сегментов не превышает $|T| \cdot (\text{среднее число сегментов на позицию})$. Однако каждый выходной переход ведёт к уникальному сегменту, и суммарное время обработки всех выходных ссылок пропорционально общему числу найденных сегментов. В худшем случае, когда шаблон состоит из

многих коротких сегментов (например, все сегменты длины 1), каждый символ текста может активировать до k выходных ссылок, так что общее время составит $O(|T| \cdot k) = O(|T| \cdot |P|) \leq 100000 \cdot 40 = 4 \cdot 10^6$, что допустимо. По памяти требуется $O(M \cdot |\Sigma|)$ для автомата и $O(|T|)$ для массива счётчиков совпадений (или $O(|P| \cdot |T|)$ при наивной реализации, но можно обойтись одним массивом размера $|T|$).

Таким образом, итоговая временная сложность — $O(|T| \cdot |P| + |P| \cdot |\Sigma|)$, а память — $O(|T| + |P| \cdot |\Sigma|)$, что эффективно при заданных ограничениях.

Тестирование

В рамках тестирования разработанных программ был создан набор unit-тестов на основе фреймворка Google Test. Для задачи точного множественного поиска набора образцов (алгоритм Ахо-Корасик с использованием суффиксных и выходных ссылок, реализация варианта 2 – подсчёт вершин и поиск пересекающихся образцов) были проверены следующие случаи:

Тест 1 (SinglePattern) для текста "TAGT", одного образца "TAG" (номер 1) выдал вхождения: (2,1). Количество вершин автомата – 4. Список пересекающихся образцов пуст. Тест пройден.

Тест 2 (MultiplePatternsNoIntersect) для текста "ACG" и образцов "A" (1), "C" (2), "G" (3) возвратил вхождения (1,1), (2,2), (3,3). Пересечений нет. Тест пройден.

Тест 3 (PatternsWithPrefixSuffix) для текста "AAA" и образцов "A", "AA", "AAA" (номера 1,2,3) дал все ожидаемые вхождения: (1,1), (1,2), (1,3), (2,1), (2,2), (3,1). Пересекающиеся образцы – 1,2,3. Количество вершин – 4. Тест пройден.

Тест 4 (IntersectingPatterns) для текста "TAGT" и образцов "TAG"(1), "AGT"(2), "G"(3) выявил вхождения (1,1), (2,2), (3,3). Все три образца пересекаются. Тест пройден.

Тест 5 (BoundaryTextLength1) для текста "A" и образцов "A", "C" выдал вхождение (1,1). Количество вершин – 3. Тест пройден.

Тест 6 (PatternLength75) для текста из 100 символов 'A' и образца из 75 символов 'A' нашёл 26 вхождений (позиции с 1 по 26). Автомат содержит 76 вершин. Тест пройден.

Тест 7 (AlphabetAllSymbols) для текста "ACGTN" и образца "ACGTN" вернул одно вхождение на позиции 1. Тест пройден.

Для задачи поиска одного образца с джокерами (алгоритм Ахо-Корасик для сегментов шаблона) были реализованы следующие тесты:

Тест 1 (NoWildcard) для шаблона "TAG", символа джокера '#' и текста "TAGT" дал вхождение на позиции 1. Тест пройден.

Тест 2 (WildcardAtEnd) для шаблона "TA?", текста "TAGTAC" и джокера '?' выдал позиции 1 и 4 (соответствует "TAG" и "TAC"). Тест пройден.

Тест 3 (WildcardAtBeginning) для шаблона "?AG" и текста "TAG" вернул позицию 1. Тест пройден.

Тест 4 (WildcardMiddle) для шаблона "A?C" и текста "ABC ATC AGC" (пробелы для разделения) нашёл вхождения на позициях 1, 5, 9. Тест пройден.

Тест 5 (MultipleSegments) для шаблона "A?C?G" и текста "AACCG" вернул единственную позицию 1. Тест пройден.

Тест 6 (NoMatch) для шаблона "T?G" в тексте "AAA" не дал ни одного вхождения. Тест пройден.

Тест 7 (BoundaryPatternLength1) для шаблона "A", джокера '?', текста "A" нашёл позицию 1. Тест пройден.

Тест 8 (PatternWithN) для шаблона "N?T", текста "NAT NBT NTT" выдал позиции 1, 5, 9. Тест пройден.

Тест 9 (OverlappingMatches) для шаблона "A?A" и текста "AAAA" вернул позиции 1 и 2. Тест пройден.

Все перечисленные тесты успешно пройдены, что подтверждает корректную работу реализованных алгоритмов, наличие выходных (сжатых) ссылок.

Вывод

В ходе выполнения лабораторной работы были успешно реализованы два алгоритма на основе метода Ахо-Корасик для решения задач множественного точного поиска и поиска с джокерами.

Первая программа находит все вхождения набора образцов в текст с временной сложностью $O(M \cdot |\Sigma| + |T| + \text{total_matches})$, где M – суммарная длина всех образцов, $|\Sigma| = 5$, $|T|$ – длина текста. Построение автомата включает суффиксные и выходные (сжатые) ссылки, что обеспечивает линейную эффективность поиска. Память составляет $O(M \cdot |\Sigma| + |T|)$. Подсчитано количество вершин в автомате, определён и выведен список образцов, имеющих пересечения с другими найденными образцами в строке поиска.

Вторая программа решает задачу поиска одного шаблона с джокерами, разбивая шаблон на максимальные безмасочные сегменты и применяя к ним автомат Ахо-Корасик. Временная сложность – $O(|T| \cdot |P| + |P| \cdot |\Sigma|)$, память – $O(|T| + |P| \cdot |\Sigma|)$, где $|P| \leq 40$. Обе программы корректно обрабатывают все символы алфавита $\{A, C, G, T, N\}$, включая редкий символ N , а также граничные случаи: пустые строки (при допустимости), образцы длины 1 и 75, текст длины 1 и 100000, перекрывающиеся вхождения, отсутствие совпадений.

Для обеих задач проведено автоматизированное тестирование с использованием Google Test. Набор тестов для первой задачи включает проверку одного образца, нескольких образцов без пересечений, образцов с префикс-суффиксными отношениями (A, AA, AAA), пересекающихся образцов (TAG, AGT, G), граничную длину текста (1 символ), максимальную длину образца (75 символов), использование всех букв алфавита (ACGTN). Для второй задачи проверены: шаблон без джокеров, джокер в конце, джокер в начале, джокер в середине, шаблон с несколькими сегментами, отсутствие вхождений, шаблон длины 1, шаблон с символом N , перекрывающиеся вхождения (A?A в AAAA). Все тесты успешно пройдены, что подтверждает

корректность работы алгоритмов в широком диапазоне входных данных, включая граничные условия, а также правильность использования выходных ссылок в автомате Ахо-Корасик.